Python E-Course

# Module 1 — Digital Image Fundamentals

*Detailed Notes*

## Module Overview

Level: Foundations

Duration: ~1 week (6-8 hours)

Prerequisites: Working Python install (see setup-guide.md)

### What You'll Learn

- Explain what a digital image actually is (grids of numbers).
- Read pixel coordinates and intensities correctly.
- Distinguish resolution from file size from color depth.
- Treat images as NumPy arrays — shape, dtype, channels.
- Navigate the (row, col) vs (x, y) coordinate confusion.

### Lessons

| # | Lesson | Duration |
|---|---|---|
| 1 | What is a Digital Image? | 25 min |
| 2 | Pixels and Coordinates | 30 min |
| 3 | Resolution and File Size | 25 min |
| 4 | Color Depth and Bit-Depth | 30 min |
| 5 | Image as a NumPy Array | 35 min |
| 6 | Coordinate Systems & Conventions | 25 min |

### Assessment

- Exercises — 18 (3 per lesson)
- Quiz — 10 MCQ
- Assignment — Image Info Inspector
- Project — Image Info Inspector CLI

### What's Next

Module 2: Image I/O & Display — load, save, and view images correctly.

# Lesson 1: What is a Digital Image?

Module: 1 — Digital Image Fundamentals

Duration: 25 minutes

Difficulty: Beginner

## Learning Objectives

- Describe a digital image as a 2D or 3D grid of intensity values.
- Distinguish grayscale from color images at the data level.
- Read a tiny matrix as an image and predict what it looks like.

## Prerequisites

- Working python + numpy + cv2.

## 1. Why This Matters

Every image processing operation is just arithmetic on a grid of numbers. Once you can stop thinking of an image as a photograph and start thinking of it as np.ndarray, everything from blurring to edge detection becomes "do this math at every pixel".

## 2. Concept

### 2.1 Grayscale = 2D grid

A grayscale image is a matrix of intensity values. In an 8-bit image, 0 = black, 255 = white, anything in between is gray.

```
A 5x5 image of a bright spot in the middle:

  10  10  10  10  10
  10  50  80  50  10
  10  80 200  80  10
  10  50  80  50  10
  10  10  10  10  10
```

Eye this matrix — you'd see a dim background with a brighter spot in the centre.

### 2.2 Color = 3D grid (height × width × channels)

A color image stacks three grayscale grids — one per color channel. Standard order in OpenCV: B, G, R (blue, green, red).

```
A 3x3 pure-blue image (BGR):

Channel 0 (Blue):     Channel 1 (Green):    Channel 2 (Red):
255 255 255           0 0 0                 0 0 0
255 255 255           0 0 0                 0 0 0
255 255 255           0 0 0                 0 0 0
```

So the array shape is (3, 3, 3) — height 3, width 3, 3 channels.

### 2.3 Continuous vs Sampled

A real-world scene is continuous; a camera samples it at a finite number of points (one per pixel). The number of samples = resolution. The precision of each sample = bit depth. We cover both in lessons 3 and 4.

### 2.4 Origin: top-left

By convention in computing, the origin (row 0, column 0) is the top-left corner. Row index grows downward; column index grows rightward.

```
        col 0  col 1  col 2
row 0  origin    →     →
row 1     ↓
row 2     ↓
```

(Math textbooks often put origin at bottom-left; image libraries don't.)

## 3. Code Examples

### Example 1: Build a tiny image from scratch

```
import numpy as np
import matplotlib.pyplot as plt

img = np.array([
    [ 10,  10,  10,  10,  10],
    [ 10,  50,  80,  50,  10],
    [ 10,  80, 200,  80,  10],
    [ 10,  50,  80,  50,  10],
    [ 10,  10,  10,  10,  10],
], dtype=np.uint8)

print("shape:", img.shape, "dtype:", img.dtype)
plt.imshow(img, cmap="gray", vmin=0, vmax=255)
plt.title("hand-crafted 5x5")
plt.axis("off")
plt.show()
```

Expected Output (visual): A 5×5 grayscale image, almost-black around the edges, with a brighter pixel in the centre (intensity 200, almost white) and a halo of mid-gray values around it. Console prints shape: (5, 5) dtype: uint8.

### Example 2: Load a real image and inspect it

```
from skimage.data import coins
import numpy as np

img = coins()
print("shape:", img.shape)        # (303, 384)
print("dtype:", img.dtype)        # uint8
```

```
print("min, max:", img.min(), img.max())
print("mean:", img.mean())
```

Expected Output (console):

```
shape: (303, 384)
dtype: uint8
min, max: 1 252
mean: 96.7...
```

What it means: A grayscale image, 303 rows × 384 cols, with darkest pixel at intensity 1 and brightest at 252. The "average" pixel is around 97 — moderately dim, consistent with the coins-on-dark-background look.

### Example 3: Build a tiny color image

```
import numpy as np
import matplotlib.pyplot as plt

img = np.zeros((4, 4, 3), dtype=np.uint8)    # BGR in OpenCV order
img[0, :] = (255,   0,   0)    # top row -> blue
img[1, :] = (  0, 255,   0)    # green
img[2, :] = (  0,   0, 255)    # red
img[3, :] = (255, 255, 255)    # white

# Display in RGB order for matplotlib
plt.imshow(img[..., ::-1])      # reverse last axis: BGR -> RGB
plt.axis("off")
plt.show()
```

Expected Output (visual): A 4×4 image: row 0 solid blue, row 1 solid green, row 2 solid red, row 3 white. (Without [..., ::-1] matplotlib would show the colours swapped because it expects RGB and we built BGR.)

## 4. Parameter Sensitivity

| Variable | Lower value effect | Higher value effect |
|---|---|---|
| Pixel intensity (uint8) | Darker (toward black) | Brighter (toward white) |
| Image shape | Fewer total samples | More samples → finer detail |
| Number of channels | 1 (grayscale only) | 3 (color), 4 (with alpha) |

## 5. Common Mistakes

| Mistake | What Happens | Fix |
|---|---|---|
| Thinking pixels are tiny squares | They're samples, not squares | Treat each value as a measurement at a point |
| Confusing the origin | Drawing flipped vertically | Top-left is (0, 0); y grows down |
| (R, G, B) assumption | Colors look wrong in OpenCV | OpenCV is BGR — see Module 2 |

## 6. Practice Tasks

- By hand: Without running code, predict what a 3×3 matrix [[255,0,0],[0,255,0],[0,0,255]] looks like.
- Build: Make a 10×10 NumPy array that's black on the left half and white on the right half. Display it.
- Inspect: Load skimage.data.camera() and print its shape, dtype, min, max, and mean.

## 7. Key Takeaways

- A grayscale image is a 2D array; a color image is 3D (H, W, 3).
- 0 = black, 255 = white in uint8.
- Origin is top-left; rows go down, columns go right.
- OpenCV color order is BGR.

## 8. What's Next

Pixels and coordinates — how to address any one pixel and how to read its value safely.

# Lesson 2: Pixels and Coordinates

Module: 1 — Digital Image Fundamentals

Duration: 30 minutes

Difficulty: Beginner

## Learning Objectives

- Access any pixel in a grayscale or color image.
- Read the value at a pixel and modify it.
- Use slicing to grab a rectangular region.
- Tell [row, col] and (x, y) apart and convert between them.

## Prerequisites

- Lesson 1: What is a Digital Image?

## 1. Why This Matters

Almost every image-processing bug at this level is a pixel-coordinate mistake. Get the indexing rules right once and they're settled forever. Get them wrong and you'll fight your own code on every project.

## 2. Concept

### 2.1 NumPy indexing: [row, col]

For a grayscale image:

```
img[r, c]
```

- r = row (0 at top)
- c = col (0 at left)

For a color image:

```
img[r, c]           # whole pixel: array of 3 values (B, G, R)
img[r, c, 0]        # blue channel at that pixel
img[r, c, 2]        # red channel
```

### 2.2 Modifying pixels

Assignment works the same way:

```
img[100, 200] = 255             # grayscale: set to white
img[100, 200] = (0, 0, 255)     # color BGR: set to red
img[100, 200, 2] = 0            # zero out only the red channel
```

### 2.3 Slicing — a rectangular region (ROI)

```
roi = img[y1:y2, x1:x2]
```

This returns rows y1 through y2-1, cols x1 through x2-1. Stop is exclusive, exactly like Python slicing on lists.

```
img[50:150, 80:240]    # 100 rows tall, 160 cols wide
```

### 2.4 Slicing is a VIEW, not a copy

This bites everyone once:

```
roi = img[50:150, 80:240]
roi[:] = 0     # also zeros img[50:150, 80:240]
```

To get an independent region:

```
roi = img[50:150, 80:240].copy()
```

### 2.5 The (x, y) convention (OpenCV drawing functions)

OpenCV's drawing functions (cv2.circle, cv2.rectangle, cv2.line, cv2.putText) expect coordinates as (x, y) tuples — x first, opposite of NumPy.

```
cv2.circle(img, (200, 100), radius=5, color=(0, 255, 0), thickness=-1)
# Draws a green dot at column 200, row 100.
# In NumPy terms: that's img[100, 200].
```

This isn't OpenCV being weird — it's the math-textbook convention of (x, y). Just keep it straight:

- Indexing/slicing arrays → [row, col] = [y, x]
- OpenCV drawing functions → (x, y)

### 2.6 Bounds checking

Out-of-bounds in NumPy raises IndexError. In OpenCV drawing functions, out-of-bounds coordinates silently clip — drawing simply happens outside the visible canvas. Useful to know when debugging "why is nothing showing up".

## 3. Code Examples

### Example 1: Read and write pixels in a real image

```
import cv2
import numpy as np
from skimage.data import camera

img = camera().copy()    # grayscale (512, 512) uint8

print("pixel at (50, 100):", img[50, 100])   # row 50, col 100

# Set a 20x20 patch in the centre to white
center_y, center_x = img.shape[0] // 2, img.shape[1] // 2
img[center_y-10:center_y+10, center_x-10:center_x+10] = 255

print("after: that patch's mean is", img[center_y-10:center_y+10, center_x-
10:center_x+10].mean())
```

Expected Output (console):

```
pixel at (50, 100): 200
after: that patch's mean is 255.0
```

Visual: Cameraman image with a small white square painted at the centre.

### Example 2: NumPy vs OpenCV — same point, two notations

```
import cv2
import numpy as np

img = np.zeros((200, 300, 3), dtype=np.uint8)

# Same point, two ways:
#   NumPy: img[100, 200]        -> row 100, col 200
#   OpenCV drawing: (200, 100)  -> x=200, y=100

img[100, 200] = (255, 255, 255)              # one white pixel via NumPy
cv2.circle(img, (200, 100), 5, (0, 0, 255), -1)  # red circle via OpenCV

# (200, 100) in OpenCV is the SAME location as [100, 200] in NumPy.
```

Expected Output (visual): A black 200×300 image with one tiny white pixel and a red 5-pixel-radius circle, both centred at the exact same spot.

### Example 3: ROI view vs copy

```
import numpy as np
img = np.arange(20).reshape(4, 5).astype(np.uint8)
print("original:\n", img)

view = img[1:3, 1:4]
view[:] = 99
print("\nafter assigning to the VIEW:\n", img)
```

Expected Output:

```
original:
 [[ 0  1  2  3  4]
  [ 5  6  7  8  9]
  [10 11 12 13 14]
  [15 16 17 18 19]]

after assigning to the VIEW:
 [[ 0  1  2  3  4]
  [ 5 99 99 99  9]
  [10 99 99 99 14]
  [15 16 17 18 19]]
```

Lesson: The view modified the original. If you needed independence, you'd have called .copy().

### Example 4: Drawing a bounding box

```
import cv2
import numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()

# Draw a green rectangle around an approximate face region:
top_left     = (180,  60)    # (x, y)
bottom_right = (340, 220)    # (x, y)
cv2.rectangle(img, top_left, bottom_right, (0, 255, 0), 3)

cv2.imwrite("astronaut_box.png", img)
```

Expected Output: Saves astronaut_box.png — the astronaut image with a 3-pixel-thick green rectangle drawn around the face area (top-left at column 180, row 60).

## 4. Parameter Sensitivity

| Parameter | Effect of small value | Effect of large value |
|---|---|---|
| Slice height (y2-y1) | Thin horizontal strip | Tall block |
| Slice width (x2-x1) | Narrow vertical strip | Wide block |
| Channel index (0/1/2) | One channel isolated | Blue/Green/Red specifically (BGR order) |

## 5. Common Mistakes

| Mistake | What Happens | Fix |
|---|---|---|
| img[x, y] instead of img[y, x] | Wrong pixel | Y first in NumPy |
| cv2.circle(img, (y, x), ...) | Circle in the wrong place | OpenCV drawing wants (x, y) |
| Forgot .copy() | Modifying ROI modifies original | Use .copy() if you need independence |
| Slicing past bounds | Empty array silently | Print img.shape first, clip indices |

## 6. Practice Tasks

- Load skimage.data.coins(). Print the pixel value at row 100, col 100.
- Set a 50×50 black square in the top-left of the same image. Display it.
- Draw a red bounding box (BGR, so red is (0,0,255)) around the top half of astronaut(). Save it.

## 7. Key Takeaways

- NumPy: img[row, col] = img[y, x] — Y first.
- OpenCV drawing: (x, y) tuples — X first.
- Slicing returns a view; .copy() for an independent array.
- Out-of-bounds: NumPy raises, OpenCV drawing silently clips.

## 8. What's Next

Resolution and file size — how the grid dimensions translate into bytes, megapixels, and quality.

# Lesson 3: Resolution and File Size

Module: 1 — Digital Image Fundamentals

Duration: 25 minutes

Difficulty: Beginner

## Learning Objectives

- Distinguish image resolution (pixels) from physical resolution (DPI).
- Predict an image's raw memory footprint from its shape and dtype.
- Distinguish raw size from file size (compression).
- Recognise when down-sampling is the right move.

## Prerequisites

- Lesson 2: Pixels and Coordinates

## 1. Why This Matters

A 24-megapixel photo can be 5 MB on disk and 70 MB in RAM. Knowing why prevents out-of-memory crashes, slow pipelines, and decisions like "should I resize this before processing?" — which you'll be doing constantly.

## 2. Concept

### 2.1 Resolution = pixel count

For an image of shape (H, W):

- Total pixels = H × W
- Megapixels (MP) = H × W / 1,000,000

A 1920 × 1080 image: 2.07 MP. A 4000 × 3000 phone photo: 12 MP.

### 2.2 DPI is something else

DPI (dots per inch) describes how big each pixel is when printed or displayed at a particular physical size. It does not change the pixel data, only the print size:

```
6000 × 4000 image at 300 DPI  -> printed at 20" × 13.3"
6000 × 4000 image at 600 DPI  -> printed at 10" × 6.7"
```

Same pixel data — just a metadata hint. For processing, ignore DPI.

### 2.3 Raw memory footprint

For a NumPy image:

```
bytes = H × W × channels × (bits-per-channel / 8)
```

Examples (using uint8 = 1 byte per channel):

| Image | Shape | Bytes | Display |
|---|---|---|---|
| HD grayscale | (1080, 1920) | 2.07 MB | uint8 |
| HD color | (1080, 1920, 3) | 6.22 MB | uint8 |
| 12 MP phone color | (3000, 4000, 3) | 36 MB | uint8 |
| Same as float32 | (3000, 4000, 3) | 144 MB | float32 |
| 4K color | (2160, 3840, 3) | 24.9 MB | uint8 |

The float32 row matters: many library functions return floats. Converting back to uint8 saves 4× memory.

### 2.4 File size = raw × (1 / compression ratio)

PNG and JPEG don't store raw arrays — they compress.

- PNG is lossless. Compression ratio depends on the image (uniform regions compress well; noisy images don't). Typical: 2-5× smaller than raw.
- JPEG is lossy. Compression ratio is dramatic — 10-30×. Trades quality for size via a quality setting (1-100).
- BMP is essentially raw + a tiny header.
- TIFF can be either.

So a 36 MB raw 12 MP photo might be a 4 MB JPEG, a 12 MB PNG, or a 36 MB BMP.

### 2.5 When to downscale

Working at full resolution is wasteful when:

- You're previewing or prototyping.
- The detail you care about (faces, license plates) is much bigger than 1 pixel.
- You're going to apply expensive operations (DFT, NLM denoising).

Rule of thumb: many image processing tasks work fine at the smallest resolution that still preserves the features you care about — often a tenth of the original.

```
small = cv2.resize(big, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
# halves both dimensions, 4x less data
```

## 3. Code Examples

### Example 1: Measure footprint

```
import numpy as np
from skimage.data import astronaut

img = astronaut()
print("shape:", img.shape)              # (512, 512, 3)
print("dtype:", img.dtype)              # uint8
print("nbytes:", img.nbytes)            # 786432 bytes
print("MB:", img.nbytes / (1024 * 1024)) # 0.75 MB
```

```
as_float = img.astype(np.float32)
print("float32 MB:", as_float.nbytes / (1024 * 1024))
```

Expected Output (console):

```
shape: (512, 512, 3)
dtype: uint8
nbytes: 786432
MB: 0.75
float32 MB: 3.0
```

What it means: 512×512×3 = 786,432 pixels-of-bytes for uint8; quadruples to 3 MB as float32. Always cast back to uint8 before saving.

### Example 2: Quality vs file size for JPEG

```
import cv2
import os
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for q in [10, 50, 90, 100]:
    fname = f"coffee_q{q}.jpg"
    cv2.imwrite(fname, img, [cv2.IMWRITE_JPEG_QUALITY, q])
    print(f"quality {q:3d} -> {os.path.getsize(fname) / 1024:6.1f} KB")
```

Expected Output (console, sizes will vary slightly):

```
quality  10 ->   8.4 KB
quality  50 ->  22.9 KB
quality  90 ->  60.7 KB
quality 100 -> 178.3 KB
```

Visual: quality 10 looks blocky and washed-out; quality 50 is acceptable for thumbnails; quality 90 is visually indistinguishable from the original at normal viewing distances.

### Example 3: Compare PNG vs JPEG vs BMP

```
import cv2
import os
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for ext in ["bmp", "png", "jpg"]:
    fname = f"coffee.{ext}"
    cv2.imwrite(fname, img)
    print(f"{ext}: {os.path.getsize(fname) / 1024:7.1f} KB")
```

Expected Output (sizes approximate):

```
bmp:   703.2 KB
png:   354.6 KB
jpg:    61.4 KB
```

### Example 4: Downscale for previews

```
import cv2
from skimage.data import astronaut

big = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
small = cv2.resize(big, None, fx=0.25, fy=0.25, interpolation=cv2.INTER_AREA)

print("big:  ", big.shape, big.nbytes // 1024, "KB")
print("small:", small.shape, small.nbytes // 1024, "KB")
```

Expected Output:

```
big:   (512, 512, 3) 768 KB
small: (128, 128, 3) 48 KB
```

Visual: thumbnail-sized 128×128 version — recognisable as the astronaut portrait, fine detail (hair strands, helmet text) lost.

## 4. Parameter Sensitivity

| Parameter | Lower | Higher |
|---|---|---|
| JPEG quality | smaller file, blocky artefacts | bigger file, no visible loss |
| Resolution (H, W) | smaller memory, less detail | bigger memory, finer detail |
| Bit depth | less precision, more banding | more precision, more memory |
| dtype | uint8 (1 byte) | float32 (4 bytes), float64 (8 bytes) |

## 5. Common Mistakes

| Mistake | What Happens | Fix |
|---|---|---|
| Loading 100 MP image into a 4 GB process | MemoryError | Downscale on load: cv2.imread(...); cv2.resize(...) |
| Saving a float image to PNG | Garbled colors or implicit cast | Convert to uint8 first with .clip(0,255).astype(np.uint8) |
| Treating JPEG-compressed = lossless | Compression artefacts accumulate | Use PNG for intermediate steps in a pipeline |

## 6. Practice Tasks

- Compute the raw size in MB for a 6000×4000 RGB uint8 image.
- Save skimage.data.coffee() as JPEG at quality 10, 50, 95. Compare file sizes and inspect quality.
- Halve the dimensions of skimage.data.astronaut() using cv2.resize with INTER_AREA. Print before/after nbytes.

## 7. Key Takeaways

- Resolution = pixel count; DPI = print size hint.
- Raw bytes = H × W × channels × bytes-per-channel.

- File size depends on format (PNG ≈ 2-5×, JPEG ≈ 10-30× smaller than raw).
- Down-sample when the detail you care about doesn't need full resolution.

## 8. What's Next

Color depth — how many bits per channel and why it matters for medical and scientific imagery.

# Lesson 4: Color Depth and Bit-Depth

Module: 1 — Digital Image Fundamentals

Duration: 30 minutes

Difficulty: Beginner

## Learning Objectives

- Define bit-depth and link it to NumPy dtypes.
- Predict the range of values for 8-bit, 16-bit, and float images.
- Choose the right dtype for medical, satellite, or HDR images.
- Convert between dtypes without losing data or banding.

## Prerequisites

- Lesson 3: Resolution and File Size

## 1. Why This Matters

A standard photo is 8 bits per channel — 256 levels. Most CT scans are 12 bits (4096 levels). Satellite imagery often 16 bits. If you load a 16-bit image and treat it as 8-bit, you'll silently throw away 8 bits of data and wonder why your contrast looks awful.

## 2. Concept

### 2.1 Bit-depth = bits per channel per pixel

| Bit-depth | Range | NumPy dtype |
|---|---|---|
| 1 | 0 or 1 | bool |
| 8 | 0–255 | uint8 |
| 16 | 0–65535 | uint16 |
| 32 (float) | typically 0.0–1.0 | float32 |
| 64 (float) | typically 0.0–1.0 | float64 |

A color image has bit-depth per channel times number of channels: a 24-bit color image = 8 bits × 3 channels.

### 2.2 256 levels is fine — until it isn't

For everyday photos, 256 gray levels per channel is far more than the eye can distinguish in any one region. Where 256 fails:

- Medical imaging — subtle tissue density differences span thousands of levels.
- Scientific imaging — telescope frames span huge dynamic ranges.
- HDR photography — captures and merges multiple exposures.
- Post-processing — heavy contrast adjustments on 8-bit reveal banding (visible stripes where smooth gradients should be).

### 2.3 Banding — what 8-bit can't do

If you stretch a low-contrast 8-bit image (say, intensities 100–110 → 0–255), you only have 11 source levels to spread over 256 target levels. The result is striped, not smooth. With a 16-bit source, you'd have 2,816 levels to work with — silky.

### 2.4 Choosing a dtype

| Use | Dtype |
|---|---|
| Web/phone photos | uint8 |
| Intermediate computation | float32 (then back to uint8) |
| Medical / scientific | uint16 |
| HDR / merged exposures | float32 |
| Masks (yes/no) | bool or uint8 (0/255) |

### 2.5 Converting between dtypes safely

uint8 → float32 (normalize to 0–1):

```
img_f = img.astype(np.float32) / 255.0
```

float32 (0–1) → uint8 (0–255):

```
img_u = (img_f * 255).clip(0, 255).astype(np.uint8)
```

uint16 → uint8 (lose precision intentionally):

```
img_u = (img_16.astype(np.float32) / 256).astype(np.uint8)
# or use cv2.normalize
img_u = cv2.normalize(img_16, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
```

uint16 → uint8 (linear stretch — preserves contrast):

```
img_u = cv2.normalize(img_16, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)
```

## 3. Code Examples

### Example 1: Demonstrate banding from over-stretching

```
import numpy as np
import matplotlib.pyplot as plt

# A near-uniform gray ramp with only 16 distinct levels
ramp = np.tile(np.linspace(100, 110, 16).astype(np.uint8), (100, 16))
print("source range:", ramp.min(), ramp.max(), "  unique:", len(np.unique(ramp)))

# Stretch 100..110 -> 0..255
stretched = ((ramp - 100) * (255 / 10)).clip(0, 255).astype(np.uint8)

plt.figure(figsize=(8, 3))
plt.subplot(121); plt.imshow(ramp, cmap="gray", vmin=0, vmax=255); plt.title("low
contrast"); plt.axis("off")
plt.subplot(122); plt.imshow(stretched, cmap="gray", vmin=0, vmax=255);
```

```
plt.title("stretched (banded)"); plt.axis("off")
plt.show()
```

Expected Output: Two grayscale strips side-by-side. The left looks almost uniformly dark gray. The right looks like 16 distinct vertical bands of gray — proof that 8-bit doesn't have enough precision to support aggressive contrast operations.

### Example 2: Visit the unique levels

```
import numpy as np
from skimage.data import coins

img = coins()
print("dtype:", img.dtype)
print("min,max:", img.min(), img.max())
print("unique levels used:", len(np.unique(img)), "out of 256 possible")
```

Expected Output:

```
dtype: uint8
min,max: 1 252
unique levels used: 240 out of 256 possible
```

### Example 3: uint8 ↔ float round-trip

```
import numpy as np

img8 = np.array([[0, 127, 255]], dtype=np.uint8)
imgF = img8.astype(np.float32) / 255.0     # 0.0, 0.498, 1.0
img8_back = (imgF * 255).clip(0, 255).astype(np.uint8)

print(img8)
print(imgF)
print(img8_back)
```

Expected Output:

```
[[  0 127 255]]
[[0.         0.49803922 1.        ]]
[[  0 127 255]]
```

### Example 4: Stretching a 16-bit image down to 8-bit

```
import cv2
import numpy as np

# Synthesize a 16-bit gradient
img16 = np.tile(np.linspace(0, 65535, 512, dtype=np.uint16), (256, 1))
print("uint16 range:", img16.min(), img16.max())

# Naive cast (loses contrast if range doesn't span 0..65535)
naive = (img16 / 256).astype(np.uint8)
print("naive uint8 range:", naive.min(), naive.max())

# Normalize: linearly stretch min..max to 0..255
```

```
stretched = cv2.normalize(img16, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)
print("stretched uint8 range:", stretched.min(), stretched.max())
```

Expected Output:

```
uint16 range: 0 65535
naive uint8 range: 0 255
stretched uint8 range: 0 255
```

For this synthetic image both give 0–255 because the source already spans full range. For a real low-contrast 16-bit image, cv2.normalize would give much better dynamic range than naive division.

## 4. Parameter Sensitivity

| Bit-depth | Memory cost | Quality after heavy edits | Best use |
|---|---|---|---|
| 1 (bool) | tiny | n/a | masks |
| 8 (uint8) | 1× | banding under heavy stretches | display / web photos |
| 16 (uint16) | 2× | smooth under heavy stretches | medical / satellite / scientific |
| 32 (float32) | 4× | excellent precision | computation, HDR |

## 5. Common Mistakes

| Mistake | What Happens | Fix |
|---|---|---|
| Saving float (0..1) as PNG without scaling | All pixels look black (clipped to 0) | Multiply by 255, clip, cast to uint8 |
| Treating uint16 as uint8 | Lose 8 bits of precision | Use cv2.normalize or proper scaling |
| Forgetting to .clip(0, 255) after float math | Wrap-around / overflow on cast | Always clip before .astype(np.uint8) |
| Long pipelines without float intermediates | Banding from repeated uint8 quantization | Work in float32 internally, cast at the end |

## 6. Practice Tasks

- Create a uint8 image full of value 200. Add 100 to it. Predict and verify the output.
- Convert that image to float32, divide by 255, add 0.4, then convert back to uint8 with clipping. Compare to task 1.
- Print the dtype, min, max, and nbytes of skimage.data.moon() (a 16-bit image after the conversion below): from skimage import img_as_uint; m = img_as_uint(skimage.data.moon()).

## 7. Key Takeaways

- Bit-depth = bits per channel per pixel.
- uint8 (256 levels) is enough for display but not for heavy editing.
- uint16 (65,536 levels) for medical / scientific data.

- Float32 for intermediate computation; cast back to uint8 to save.
- Always clip then cast when going from float to uint8.

## 8. What's Next

Treating an image as a NumPy array — the fluent ops you'll use every day.

# Lesson 5: Image as a NumPy Array

Module: 1 — Digital Image Fundamentals

Duration: 35 minutes

Difficulty: Beginner

## Learning Objectives

- Apply NumPy operations directly to images.
- Use broadcasting for per-pixel and per-channel ops.
- Apply boolean masks to images.
- Build a small image from scratch with np.zeros/np.full/np.tile/np.arange.

## Prerequisites

- Lesson 4: Color Depth and Bit-Depth
- Basic NumPy familiarity. Refer to NumPy-for-Images cheatsheet as needed.

## 1. Why This Matters

NumPy IS the lingua franca of image processing in Python. OpenCV functions take and return NumPy arrays. scikit-image is built on NumPy. matplotlib displays NumPy arrays. Knowing the dozen NumPy patterns below replaces hundreds of lines of for-loops over pixels.

## 2. Concept

### 2.1 Vectorized arithmetic

Operate on the whole image at once:

```
img + 50              # brighten every pixel by 50 (watch overflow!)
img * 1.2             # scale; returns float
255 - img             # photographic negative
```

Always remember: + 50 on uint8 wraps around (200 + 100 = 44). Use cv2.add for saturating arithmetic, or cast to float first.

### 2.2 Broadcasting

Combine arrays of different shapes if axes line up:

```
img.shape          # (H, W, 3)
scale = np.array([1.0, 0.5, 0.5])    # (3,)  - tints toward blue (BGR)
out = (img.astype(np.float32) * scale).clip(0, 255).astype(np.uint8)
```

Broadcasting matched the (3,) along the channel axis — equivalent to multiplying each channel separately, but in one line.

### 2.3 Boolean masks

```
mask = img > 128            # bool array shape (H, W) or (H, W, 3)
img[mask] = 255             # set bright pixels to max
print(mask.sum())           # how many pixels qualified
```

For per-pixel masks on color images, often build the mask from one channel:

```
gray = img.mean(axis=2)
bright = gray > 200          # (H, W) bool
```

### 2.4 Reductions (collapsing axes)

```
img.mean()                   # one scalar: overall brightness
img.mean(axis=(0, 1))        # per-channel mean: shape (3,)
img.mean(axis=2)             # collapse channels -> grayscale shape (H, W)
img.sum(axis=0)              # column-wise sum: shape (W,)
```

Useful for histograms, projections, and feature stats.

### 2.5 Building images from scratch

```
black = np.zeros((100, 100, 3), dtype=np.uint8)
white = np.full((100, 100), 255, dtype=np.uint8)
gray_50 = np.full((100, 100), 128, dtype=np.uint8)

# horizontal gradient 0..255
gradient_h = np.tile(np.arange(256, dtype=np.uint8), (100, 1))    # shape (100, 256)
# vertical gradient
gradient_v = np.tile(np.arange(256, dtype=np.uint8)[:, None], (1, 100))

# random noise
noise = (np.random.rand(100, 100) * 255).astype(np.uint8)
```

### 2.6 Concatenation

```
np.hstack([a, b])     # side-by-side  (heights must match)
np.vstack([a, b])     # stacked       (widths must match)
```

Very handy for "before / after" displays.

## 3. Code Examples

### Example 1: Photographic negative

```
import cv2
import numpy as np
from skimage.data import camera

img = camera()
neg = 255 - img

vis = np.hstack([img, neg])
cv2.imwrite("camera_vs_negative.png", vis)
print("saved:", vis.shape, vis.dtype)
```

Expected Output (console): saved: (512, 1024) uint8

Visual: the cameraman image on the left, his photographic negative on the right (bright sky becomes dark, dark coat becomes light).

### Example 2: Brighten safely (no overflow)

```
import cv2
import numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)

# Wrong: wraps around
wrong = img + 80           # uint8 overflow!

# Right: saturating add
right = cv2.add(img, np.full_like(img, 80))

print("wrong sample pixels (B,G,R):", wrong[0, 0])
print("right sample pixels (B,G,R):", right[0, 0])
```

Expected Output (console, exact values depend on the pixel):

```
wrong sample pixels (B,G,R): [22 33 12]    <- looks darker than original due to
wrap
right sample pixels (B,G,R): [255 255 92]  <- saturates at 255
```

### Example 3: Boolean mask — keep only bright pixels

```
import numpy as np
from skimage.data import coins
import matplotlib.pyplot as plt

img = coins()
bright = img > 150

out = np.zeros_like(img)
out[bright] = img[bright]

print("bright pixels:", bright.sum(), "out of", img.size)
plt.figure(figsize=(8, 4))
plt.subplot(121); plt.imshow(img, cmap="gray"); plt.axis("off");
plt.title("original")
plt.subplot(122); plt.imshow(out, cmap="gray"); plt.axis("off"); plt.title("bright
only")
plt.show()
```

Expected Output: Two panels — original coin image, and a version where only pixels brighter than 150 are kept (the coins themselves; background is black).

### Example 4: Pure-NumPy grayscale conversion

```
import numpy as np
from skimage.data import astronaut

img = astronaut()          # (512, 512, 3) RGB
```

```
# Weighted gray: 0.299 R + 0.587 G + 0.114 B (ITU-R BT.601)
weights = np.array([0.299, 0.587, 0.114], dtype=np.float32)
gray = (img.astype(np.float32) * weights).sum(axis=2)
gray = gray.clip(0, 255).astype(np.uint8)

print(img.shape, "->", gray.shape, gray.dtype)
```

Expected Output:

```
(512, 512, 3) -> (512, 512) uint8
```

Visual: monochrome version of the astronaut portrait. (cv2 does this with cv2.cvtColor(img, cv2.COLOR_RGB2GRAY) in one line — but it's good to see what's actually happening inside.)

### Example 5: Build a checkerboard from scratch

```
import numpy as np
import matplotlib.pyplot as plt

H, W, square = 200, 200, 25
rows = (np.arange(H) // square) % 2
cols = (np.arange(W) // square) % 2
board = ((rows[:, None] ^ cols[None, :]) * 255).astype(np.uint8)

print(board.shape, board.dtype, "unique vals:", np.unique(board))
plt.imshow(board, cmap="gray"); plt.axis("off"); plt.show()
```

Expected Output:

```
(200, 200) uint8 unique vals: [  0 255]
```

Visual: an 8×8 checkerboard, 25-pixel squares of pure black and pure white.

## 4. Parameter Sensitivity

| Op | Small | Large |
|---|---|---|
| img + N | small brightness boost | wraps around (uint8) — switch to cv2.add |
| img * f | dims toward black | brightens but produces float >255 |
| Mask threshold | many pixels selected | few pixels selected |

## 5. Common Mistakes

| Mistake | What Happens | Fix |
|---|---|---|
| img + 100 on uint8 | overflow / wrap | cv2.add or cast to float, clip, cast back |
| for r in range(H): for c in range(W): ... | painfully slow | use vectorized ops or broadcasting |
| Forgetting axis= on reductions | scalar when you wanted an array | specify the axis: axis=2 collapses channels |
| Wrong broadcast shape | ValueError | print shapes and align — (3,) |

| | | |
|---|---|---|
| | | works against (H, W, 3) |

## 6. Practice Tasks

- Invert (negative of) skimage.data.astronaut() and display.
- Convert that astronaut to grayscale using only NumPy (the weighted-sum trick).
- Build a 256×256 horizontal gradient using np.tile and np.arange.

## 7. Key Takeaways

- Operate on whole images, not pixel-by-pixel.
- Broadcasting lets you apply per-channel or per-row ops without loops.
- Boolean masks select pixels by condition.
- Build images from scratch with zeros/full/tile/arange/random.
- Stack with hstack/vstack for side-by-side displays.

## 8. What's Next

The last lesson — pinning down coordinate conventions across libraries so you stop swapping x and y.

# Lesson 6: Coordinate Systems & Conventions

Module: 1 — Digital Image Fundamentals

Duration: 25 minutes

Difficulty: Beginner

## Learning Objectives

- Translate fluently between NumPy [row, col] and Cartesian (x, y).
- Use OpenCV functions that expect (x, y) tuples without confusion.
- Identify which library uses BGR vs RGB.
- Know skimage's (row, col) everywhere convention.

## Prerequisites

- All previous Module-1 lessons.

## 1. Why This Matters

This one lesson exists because the single most common image-processing bug is swapping x and y or BGR and RGB. Once these are settled, you save weeks across your career.

## 2. Concept

### 2.1 The cheat sheet

| Library / API | Pixel access | Point as argument | Color order |
|---|---|---|---|
| NumPy indexing | img[row, col] | n/a | depends on source |
| OpenCV drawing/geometry | n/a | (x, y) tuple | BGR |
| OpenCV cv2.warpAffine | n/a | (W, H) for dsize | BGR |
| OpenCV cv2.findContours | returns (x, y) arrays | — | — |
| scikit-image | img[row, col] | (row, col) | RGB |
| matplotlib imshow | — | uses array as-is | expects RGB |
| Pillow | img.getpixel((x, y)) | (x, y) | RGB |

### 2.2 Why this exists

- NumPy descends from math/scientific arrays — rows first, mirroring A[i][j] in linear algebra.
- OpenCV descends from computer graphics — (x, y) matches the way you'd write coordinates in geometry.
- scikit-image chose to align with NumPy throughout — (row, col) everywhere, including arguments to its own functions.

There's no winning here. There's only knowing which library you're calling.

### 2.3 Visualising the difference

```
A 4×5 image:

NumPy view (img[row, col]):

       col 0  col 1  col 2  col 3  col 4
row 0   a      b      c      d      e
row 1   f      g      h      i      j
row 2   k      l      m      n      o
row 3   p      q      r      s      t

To address 'm' (row 2, col 2):
  NumPy:           img[2, 2]
  OpenCV point:    (2, 2)    -> x=2, y=2  -> same location, same numbers (by luck
of equal values)

To address 'i' (row 1, col 3):
  NumPy:           img[1, 3]
  OpenCV point:    (3, 1)    -> x=3, y=1  -> swap!
```

When the row and column happen to be equal, both notations give the same tuple. That makes it harder to notice when you've swapped them. Don't rely on test cases where row == col.

### 2.4 Common conversion mistakes

```
# OpenCV docs say: cv2.line(img, pt1, pt2, color, thickness)
# pt1 and pt2 are (x, y).

h, w = img.shape[:2]

# Wrong (swapped):
cv2.line(img, (h, w), (0, 0), (0, 255, 0), 2)

# Right:
cv2.line(img, (w, h), (0, 0), (0, 255, 0), 2)   # bottom-right to top-left
```

### 2.5 Width / height in function arguments

cv2.resize(img, dsize) — dsize is (width, height), not (height, width).

```
# Resize to 200 wide, 100 tall:
resized = cv2.resize(img, (200, 100))    # (W, H)

# But the resulting array's shape is:
resized.shape    # (100, 200, ...) -> (H, W, …)
```

### 2.6 BGR vs RGB cheat sheet

| Source | Order |
|---|---|
| cv2.imread | BGR |
| cv2.imwrite (input) | BGR |
| cv2.cvtColor(img, cv2.COLOR_BGR2RGB) | swap to RGB |
| skimage.io.imread | RGB |
| skimage.data.* | RGB |

| | |
|---|---|
| PIL.Image.open(...).convert("RGB") | RGB |
| matplotlib.pyplot.imshow | expects RGB |

So the typical "load with OpenCV, display with matplotlib" recipe is:

```
img_bgr = cv2.imread("x.jpg")
plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
```

Skip the conversion and your photo will look weirdly blue-shifted (red and blue channels swapped).

## 3. Code Examples

### Example 1: Same point, three notations

```
import cv2
import numpy as np

img = np.zeros((200, 300, 3), dtype=np.uint8)  # H=200, W=300

# Target: row 75, col 150  (i.e. y=75, x=150)
img[75, 150] = (255, 255, 255)                          # NumPy
cv2.circle(img, (150, 75), 6, (0, 255, 0), 1)           # OpenCV (x, y)
cv2.putText(img, "*", (150, 75), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255))

cv2.imwrite("same_point.png", img)
print("img shape (H,W,C):", img.shape)
```

Expected Output (console): img shape (H,W,C): (200, 300, 3)

Visual: at column 150, row 75 there's a single white pixel, a thin green circle around it, and a red asterisk nearby — all marking the same location, addressed three different ways.

### Example 2: BGR vs RGB display

```
import cv2
import matplotlib.pyplot as plt
from skimage.data import astronaut

# skimage returns RGB
img_rgb = astronaut()

# Convert to BGR (what cv2.imread would give):
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)

# Display both with matplotlib (which expects RGB):
plt.figure(figsize=(8, 4))
plt.subplot(121); plt.imshow(img_rgb); plt.title("RGB (correct)"); plt.axis("off")
plt.subplot(122); plt.imshow(img_bgr); plt.title("BGR shown as RGB (wrong)");
plt.axis("off")
plt.show()
```

Expected Output (visual): Two panels. Left: the astronaut portrait in natural colors. Right: the same astronaut with the orange / red space-suit elements showing up as blue / cyan — because matplotlib interpreted the B channel as R.

### Example 3: cv2.resize argument order

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
print("original:", img.shape)                    # (512, 512, 3) -> (H, W, C)

# Resize to width=300, height=100
small = cv2.resize(img, (300, 100))              # dsize is (W, H)
print("resized: ", small.shape)                  # (100, 300, 3) -> (H, W, C)
```

Expected Output:

```
original: (512, 512, 3)
resized:  (100, 300, 3)
```

### Example 4: scikit-image is row-col throughout

```
from skimage.draw import circle_perimeter
from skimage.data import camera
import numpy as np

img = camera().copy()
# skimage draws use (row, col)
rr, cc = circle_perimeter(r=100, c=200, radius=30)
img[rr, cc] = 255

print("shape:", img.shape, "modified pixels:", len(rr))
```

Expected Output: shape: (512, 512) modified pixels: ~189 and a white circle of radius 30 centred at row 100, col 200 of the cameraman image. Note that arguments are r= (row) and c= (col) — same convention as NumPy indexing.

## 4. Parameter Sensitivity

(This lesson is about conventions, not parameters — no sensitivity table.)

## 5. Common Mistakes

| Mistake | What Happens | Fix |
|---|---|---|
| cv2.circle(img, (y, x), ...) | Circle in wrong place | OpenCV is (x, y) |
| cv2.resize(img, (H, W)) | Image gets stretched | cv2.resize(img, (W, H)) |
| Skipping cv2.cvtColor(img, COLOR_BGR2RGB) before plt.imshow | Reds look blue | Always convert before displaying with matplotlib |
| Mixing skimage and cv2 without converting | Channels swap silently | Pick one stack; convert at boundaries |
| Treating returned | Plot ends up rotated | They're (x, y); use pt[1] for |

| | | |
|---|---|---|
| cv2.findContours points as (row, col) | | row, pt[0] for col when indexing |

## 6. Practice Tasks

- Load astronaut() with skimage (RGB). Convert to BGR and save with cv2.imwrite. Reload with cv2.imread. Display correctly in matplotlib.
- Draw a cv2.rectangle from (50, 30) to (150, 100). Predict what the rectangle's corners look like in [row, col] terms.
- Resize an image to width=400, height=200 with cv2.resize. Print the resulting shape.

## 7. Key Takeaways

- NumPy / skimage: (row, col).
- OpenCV drawing & geometry: (x, y).
- cv2.resize(..., (W, H)) — argument order is opposite of .shape.
- OpenCV is BGR; matplotlib/skimage are RGB.
- Convert at library boundaries; don't mix conventions in one expression.

## 8. What's Next

End of Module 1. Module 2: Image I/O & Display — actually load and save images correctly across formats.

# Module 1 — Exercises

18 exercises (3 per lesson).

## 1.1 What is a Digital Image?

- (E) Hand-decode: Without code, sketch what the 4×4 array [[0,0,0,0],[0,255,255,0],[0,255,255,0],[0,0,0,0]] looks like.
- (M) Stats: Load skimage.data.camera(). Print shape, dtype, min, max, mean, and total pixel count.
- (H) Build a color image: Make a (50, 50, 3) uint8 BGR image whose top half is red and bottom half is blue. Save as PNG.

## 1.2 Pixels and Coordinates

- (E) Set img[50, 100] of skimage.data.coins() to 255 and print it before and after.
- (M) Draw a green rectangle with cv2.rectangle from (50, 30) to (150, 100) on a 200×200 black image. Save it.
- (H) Demonstrate ROI-is-a-view: build a 4×5 array, slice the middle 2×3 region, modify the slice, print the original. Then redo with .copy() and compare.

## 1.3 Resolution and File Size

- (E) Compute raw uint8 bytes for shapes (1024,768), (1920,1080,3), (4000,3000,3). Print in MB.
- (M) Save skimage.data.coffee() as JPEG at qualities 10/50/95. Compare file sizes.
- (H) Halve dimensions of astronaut() with cv2.resize(..., INTER_AREA). Compare nbytes before/after.

## 1.4 Color Depth

- (E) Predict what np.array([200], dtype=np.uint8) + np.array([100], dtype=np.uint8) returns. Verify.
- (M) Convert coins() to float32 in [0, 1], multiply by 1.3, clip, convert back to uint8. Compare to original.
- (H) Build a uint16 gradient np.linspace(0, 65535, 256) tiled into a 100×256 image. Convert to uint8 two ways: naive (/256) and cv2.normalize. Compare visually.

## 1.5 Image as NumPy Array

- (E) Build a 256×256 horizontal gradient using np.tile + np.arange. Save as PNG.
- (M) Convert astronaut() to grayscale using only NumPy (weighted-sum, no cv2.cvtColor).
- (H) Build a 200×200 checkerboard with 25×25 squares. No cv2. Save as PNG.

## 1.6 Coordinate Systems

- (E) Predict where cv2.circle(img, (40, 70), 5, (0,255,0), -1) draws — row and col, in NumPy terms.
- (M) Resize astronaut() to width=200, height=50. Print resulting shape.

- (H) Load chelsea() (skimage RGB), save as PNG with cv2.imwrite, reload with cv2.imread, display correctly with matplotlib using cvtColor.

## Module 1 — Quiz

10 MCQs covering all 6 lessons.

### Q1

What's the shape of a 1920×1080 BGR color image as a NumPy array? A. (1920, 1080, 3)  B. (1080, 1920, 3)  C. (1080, 1920)  D. (3, 1080, 1920)

Answer: B — shape is (H, W, C) and H is the first axis. (L1, L5)

### Q2

How do you access the pixel at row 50, col 200 of a NumPy image? A. img[200, 50]  B. img[50, 200]  C. img[(50, 200)]  D. Both B and C

Answer: D — both work; B is clearer. (L2)

### Q3

What argument order does cv2.circle(img, center, radius, ...) expect for center? A. (row, col)  B. (x, y)  C. (col, row)  D. (y, x)

Answer: B (= C). (L2, L6)

### Q4

What does np.array([200], dtype=np.uint8) + np.array([100], dtype=np.uint8) return? A. [300]  B. [44]  C. [255]  D. error

Answer: B — uint8 wraps around. Use cv2.add for saturation. (L4, L5)

### Q5

A 12 MP color image as uint8 occupies roughly how much RAM as a raw NumPy array? A. 4 MB  B. 12 MB  C. 36 MB  D. 144 MB

Answer: C — 4000×3000×3 = 36,000,000 bytes ≈ 36 MB. (L3)

### Q6

Which of these is lossless? A. JPEG  B. PNG  C. WebP (default)  D. JPEG XR

Answer: B (L3)

### Q7

You're displaying a cv2.imread-loaded image with plt.imshow. Colors look wrong. The fix is: A. Save and reload  B. cv2.cvtColor(img, cv2.COLOR_BGR2RGB) before imshow  C. Normalize  D. Use cmap='gray'

Answer: B — OpenCV is BGR; matplotlib expects RGB. (L6)

## Q8

Slicing roi = img[10:50, 20:80] creates: A. A copy of the region  B. A view into the original  C. A list of pixels  D. A bool mask

Answer: B — modifying roi modifies img. Use .copy() for independence. (L2)

## Q9

Which dtype gives 65,536 levels per channel? A. uint8  B. uint16  C. float32  D. int8

Answer: B (L4)

## Q10

cv2.resize(img, dsize) — what does dsize mean? A. (height, width)  B. (width, height)  C. (channels, width)  D. (rows, cols)

Answer: B — opposite of .shape order. (L6)

# Module 1 — Assignment: Image Info Inspector

Estimated time: 1.5 hours

Combines concepts from: Module 1 (all lessons)

## Brief

Write a program that takes an image path and prints a complete profile: dimensions, dtype, channel count, memory footprint, raw pixel statistics per channel, and then displays each channel separately.

## Requirements

The program must:

- Accept a file path (CLI arg or hard-coded).
- Load the image with cv2.imread (and check for None).
- Print:
- File path
- File size on disk (bytes / KB)
- Image shape (H, W) or (H, W, 3)
- dtype
- Megapixels (computed from H × W)
- Raw NumPy footprint in MB
- For grayscale: min, max, mean, std
- For color (BGR): per-channel min, max, mean, std — labelled B, G, R
- Display each channel separately as a grayscale image (matplotlib or save to disk):
- 3 subplots labelled "Blue", "Green", "Red" for color images
- Plus the original image (converted to RGB) on a 4th subplot for reference
- Output a clean text report and a single PNG visualisation (report.png).

## Constraints

- Use Module 1 concepts only.
- No loops over pixels — use vectorised NumPy.
- Use matplotlib to save / display.
- BGR conventions must be respected in labels.

## Expected Output (sample)

```
== Image Info Report ==
Path        : datasets/starter/astronaut.png
File size   : 410.5 KB
Shape       : (512, 512, 3)
dtype       : uint8
Megapixels  : 0.26 MP
Raw memory  : 0.75 MB
```

```
Per-channel statistics (BGR):
  B  min=  0  max=255  mean= 73.4  std= 60.1
  G  min=  0  max=255  mean= 78.2  std= 53.7
  R  min=  0  max=255  mean=125.4  std= 71.0

Saved report visualisation -> report.png
```

report.png is a 2×2 grid: original (top-left), blue / green / red channel as grayscale (top-right, bottom-left, bottom-right).

## Hints

- File size: os.path.getsize(path) or Path(path).stat().st_size.
- Per-channel stats: img[..., c].mean(), img[..., c].std(), etc.
- Display order matters: matplotlib imshow(channel, cmap='gray', vmin=0, vmax=255) for fair comparison.

## Grading Rubric

| Criterion | Weight |
|---|---|
| Loads + None check | 15% |
| Correct stats + units | 30% |
| Visualisation correct | 25% |
| BGR labels correct | 15% |
| Style + naming | 15% |

## Submission

Save as assignment_module1.py.

## Module 1 — Mini Project: Image Info Inspector CLI

### Overview

A polished command-line tool that wraps the assignment into something you'd actually use as a developer. Takes one or more images, prints a report, and optionally exports a JSON summary.

### Concepts Used

- File I/O with cv2.imread (M1 L5)
- NumPy stats (M1 L5)
- File-size & dtype reasoning (M1 L3, L4)
- Coordinate conventions (M1 L6)

### Features

- imginfo PATH [PATH ...] — print a one-block report for each image.
- --json FILE — append every report as a JSON object.
- --quiet — only print one line per image (path, shape, size).
- --per-channel (default for color images) — show channel stats.

### Starter Code

imginfo.py:

```
import argparse
import json
import os
import sys
from pathlib import Path

import cv2
import numpy as np


def inspect(path: str) -> dict:
    if not os.path.exists(path):
        raise FileNotFoundError(path)

    img = cv2.imread(path, cv2.IMREAD_UNCHANGED)
    if img is None:
        raise ValueError(f"OpenCV couldn't decode: {path}")

    file_bytes = os.path.getsize(path)
    h = img.shape[0]
    w = img.shape[1]
    channels = 1 if img.ndim == 2 else img.shape[2]
    nbytes = img.nbytes
    info = {
        "path": str(Path(path).resolve()),
        "file_bytes": file_bytes,
        "shape": list(img.shape),
```

```
        "dtype": str(img.dtype),
        "channels": channels,
        "megapixels": round(h * w / 1_000_000, 3),
        "raw_mb": round(nbytes / (1024 * 1024), 3),
    }

    if channels == 1:
        info["stats"] = {
            "min": int(img.min()),
            "max": int(img.max()),
            "mean": round(float(img.mean()), 2),
            "std": round(float(img.std()), 2),
        }
    else:
        labels = ["B", "G", "R", "A"][:channels]    # OpenCV order
        per_channel = {}
        for i, label in enumerate(labels):
            ch = img[..., i]
            per_channel[label] = {
                "min": int(ch.min()),
                "max": int(ch.max()),
                "mean": round(float(ch.mean()), 2),
                "std": round(float(ch.std()), 2),
            }
        info["stats"] = per_channel

    return info


def fmt_report(info: dict) -> str:
    lines = [
        "== Image Info Report ==",
        f"Path        : {info['path']}",
        f"File size   : {info['file_bytes'] / 1024:.1f} KB",
        f"Shape       : {tuple(info['shape'])}",
        f"dtype       : {info['dtype']}",
        f"Channels    : {info['channels']}",
        f"Megapixels  : {info['megapixels']} MP",
        f"Raw memory  : {info['raw_mb']} MB",
    ]
    stats = info["stats"]
    if "mean" in stats:
        s = stats
        lines.append(f"Stats       : min={s['min']:3d}  max={s['max']:3d}
mean={s['mean']:6.2f}  std={s['std']:6.2f}")
    else:
        lines.append("Per-channel statistics:")
        for label, s in stats.items():
            lines.append(
                f"  {label}  min={s['min']:3d}  max={s['max']:3d}  "
                f"mean={s['mean']:6.2f}  std={s['std']:6.2f}"
            )
    return "\n".join(lines)
```

```
def main(argv=None):
    p = argparse.ArgumentParser(prog="imginfo", description="Inspect image files.")
    p.add_argument("paths", nargs="+", help="image file paths")
    p.add_argument("--json", metavar="FILE", help="append all results as JSON")
    p.add_argument("--quiet", action="store_true", help="one line per file")
    args = p.parse_args(argv)

    all_info = []
    for path in args.paths:
        try:
            info = inspect(path)
        except (FileNotFoundError, ValueError) as e:
            print(f"[error] {path}: {e}", file=sys.stderr)
            continue
        all_info.append(info)
        if args.quiet:
            print(f"{path:40s}  {tuple(info['shape'])}
{info['file_bytes']/1024:.1f} KB")
        else:
            print(fmt_report(info))
            print()

    if args.json:
        with open(args.json, "w", encoding="utf-8") as f:
            json.dump(all_info, f, indent=2)
        print(f"saved -> {args.json}")


if __name__ == "__main__":
    main()
```

## Expected Interaction

```
$ python imginfo.py datasets/starter/astronaut.png
== Image Info Report ==
Path        : D:\Image Processing e-course\datasets\starter\astronaut.png
File size   : 410.5 KB
Shape       : (512, 512, 3)
dtype       : uint8
Channels    : 3
Megapixels  : 0.262 MP
Raw memory  : 0.75 MB
Per-channel statistics:
  B  min=  0  max=255  mean= 73.40  std= 60.10
  G  min=  0  max=255  mean= 78.20  std= 53.70
  R  min=  0  max=255  mean=125.40  std= 71.00

$ python imginfo.py datasets/starter/*.png --quiet --json all.json
datasets/starter/astronaut.png                  (512, 512, 3)  410.5 KB
datasets/starter/brick.png                      (512, 512)     223.4 KB
datasets/starter/camera.png                     (512, 512)     258.9 KB
```

```
... etc
saved -> all.json
```

## Extension Challenges

- Add an EXIF reader (use Pillow's ExifTags) for JPEGs.
- Add a --save-channels DIR flag that writes B/G/R as separate grayscale PNGs.
- Add a histogram summary (with text bars showing distribution per channel).

## Grading Rubric

| Criterion | Weight |
|---|---|
| All commands work | 30% |
| Correct stats | 25% |
| JSON output valid | 15% |
| Friendly error msgs | 15% |
| Style + functions | 15% |